AI Property Valuation Assistant

A 6–week data-science project. You will build, compare, and evaluate two machine-learning models that predict house prices from real data — then add an AI layer that explains each prediction in plain English and understands a house described in words.

`5 students · 1 team` `Mentor: Dev Dutta` `Kickoff: Tue 23 Jun 2026`
`Report due: 31 Jul 2026` `Meetings: Tue 7–9 PM IST · Teams`

01

The project, in one page

The problem. What is a house worth? Buyers, sellers, and banks all need an estimate — and the honest answer comes from data: similar houses, their features, and what they sold for. Your system learns that relationship from ~2,900 real home sales and predicts the price of a house from its details. Then it adds a friendly AI layer that *explains* the estimate and lets a user describe a house in plain English.

What you will deliver

1. **A Jupyter notebook** that builds *two* prediction models (a Linear Regression baseline and a Random Forest), **compares them**, and **picks the better one** with proper metrics. *This is the graded core.*
2. **A working valuation "app"** — using the chosen model to price houses it has never seen: a *single* house from its features, or a whole *batch* of houses at once. *This is the payoff — the reason you built the model.*
3. **An AI assistant layer** — a free LLM (Groq) that explains a predicted price in plain English (one house or a batch) and parses a plain-English house description into features the model can price.
4. **(Optional, only if time allows)** a tiny interactive demo UI that wraps the app — in-notebook with `ipywidgets`.

5. **The Project Report** — the primary assessed artifact (see §07).

The one thing that makes this Data Science, not a chatbot: the prediction models — building two, comparing them, picking a winner, and then **using it to price real houses**. Everything else supports that.

Why build a price predictor at all? So you can value a house *nobody has sold yet* — instantly, from its features. That's the product a bank, buyer, or agent would actually use. Modules 1–7 build and choose the model; **Module 8 is where it pays off** — you type in a house and get a price.

The honesty rule you'll hear all project (and it earns marks): the **machine-learning model predicts the price**; the **LLM only explains it** in words and reads the user's description. A language model can't fairly predict a number from a table — so we never claim it does. Say this clearly in your report.

Scope — read this carefully

✅ IN SCOPE	🚫 OUT OF SCOPE (DELIBERATELY)
Real housing dataset · cleaning · EDA · two models (Linear Regression + Random Forest) · proper evaluation · feature importance · charts · an LLM explain + describe-a-house layer · optional tiny demo	A live website / deployed app · user accounts · scraping live listings · predicting Indian/Bengaluru prices · trying every model under the sun · an LLM that "predicts" prices

Why so much is out of scope: ~6 weeks, remote team, first real project. We deliberately remove the parts where student projects usually collapse (deployment, scraping, scope creep) so you can go *deep* on the data science and finish with a strong report. Ambition lives in the *evaluation*, not in the plumbing.

02

The data (real, verified)

You use **one** dataset — your mentor gives it to you, so there is nothing to hunt for. Your Week-1 job is to load it, look at it, and confirm its shape — don't assume, verify.

The dataset — the Ames housing data

This is the **Ames, Iowa housing dataset**, compiled by Prof. Dean De Cock as a modern, richer alternative to the old Boston housing data. It has **2,930 home sales** and **82 columns** describing each house — size, quality, year built, garage, basement, neighbourhood, and the `SalePrice` you predict. Your mentor shares the file `AmesHousing.csv` on Teams.

Reference: Dean De Cock, "Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project," *Journal of Statistics Education*, 2011. The same data underpins the well-known Kaggle "House Prices: Advanced Regression Techniques" competition. We start from a hand-picked **14-feature subset** (see the feature dictionary we give you), then you may expand it.

De-risking move: get the whole pipeline running end-to-end on the 14-feature subset *first* — load → clean → two models → compare. Only once that works should you add more features and see if accuracy improves. A working simple version beats an ambitious broken one.

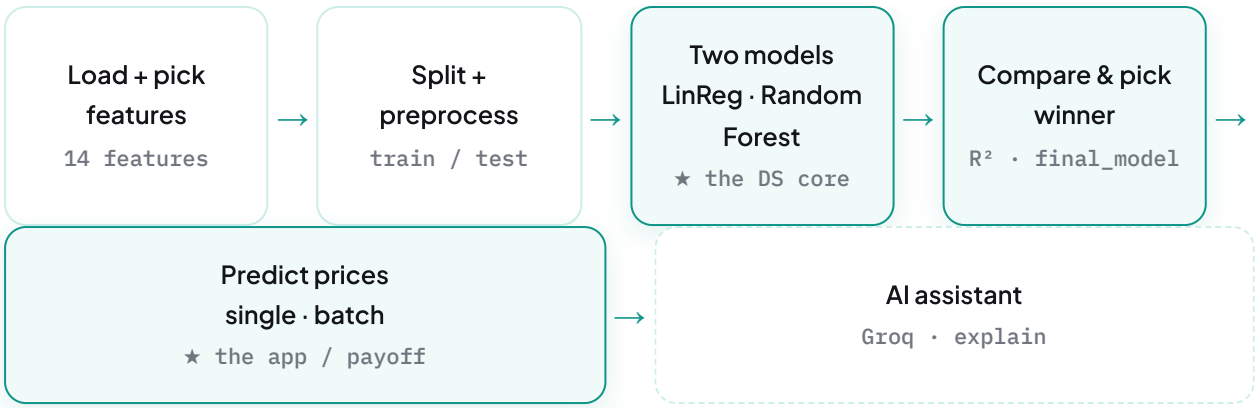
One ethics footnote for your report: the famous old "Boston Housing" dataset was removed from scikit-learn because it contained a racially-biased feature. The Ames data is the modern replacement — and a good place to note in your report that data choices carry ethical weight.

03

What you're building

Everything below runs **inside one Jupyter notebook** (top to bottom). The optional demo UI stays in the notebook (`ipywidgets`).

Your step-by-step build manual: this Handbook explains *what* and *why*. For the exact *how* — every module's inputs and outputs, runnable starter code, worked examples, and common errors — work from the **Build Specification →**. Read it alongside this page.



The same flow, with a real example

Take **one house**: *3 bedrooms, about 1,500 sq ft, built in 2005, overall quality 7, a 2-car garage, 2 full bathrooms.* Here is what the system does with it — and where the two real questions get answered: *what is it worth?* and *why?*

1. **Learn from history.** The models train on ~2,900 past sales — what size, quality, and location did to price.
2. **Predict.** The winning model (Random Forest) prices this house at **≈ $188,000**.
3. **Explain.** The LLM writes: *"This sits in the mid-range largely because its overall quality (7/10) and ~1,500 sq ft of living space are solid but not premium; the 2005 build and 2-car garage add value…"* — using the model's *actual* top drivers, and **never inventing a different price**.

The two models, concretely

Baseline — Linear Regression (a straight-line fit; the yardstick):

```
# preprocessor (from M4) + the model, in one pipeline that cleans then fits
lr_model = Pipeline([("prep", preprocessor), ("model", LinearRegression())])
lr_model.fit(X_train, y_train)
r2 = r2_score(y_test, lr_model.predict(X_test))   # ~0.88
```

Smarter — Random Forest (hundreds of decision trees, averaged):

```
rf_model = Pipeline([("prep", preprocessor),
                     ("model", RandomForestRegressor(n_estimators=300, random_sta
rf_model.fit(X_train, y_train)
r2 = r2_score(y_test, rf_model.predict(X_test))   # ~0.92 - it wins
```

You then **compare** the two: which predicts real prices more accurately, and why? That comparison is the heart of the report.

The AI assistant (verified Groq quickstart)

```
# from console.groq.com/docs/quickstart - free tier, no credit card
from groq import Groq
client = Groq(api_key=os.environ["GROQ_API_KEY"])
resp = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role":"user","content":
        f"A model priced this house at ${price:,.0f}. Do NOT change the price. "
        f"Explain in 3-4 sentences why, using: {drivers}."}])
print(resp.choices[0].message.content)
```

The AI layer is an **add-on, not load-bearing**. If the Groq call fails on demo day, the prediction study still stands on its own. That is by design.

Library map — most of this is new, and that's the point

COMPONENT	PYTHON LIBRARY	
Loading & cleaning data	`pandas`, `numpy`	familiar
Charts (EDA + results)	`matplotlib`	familiar
Split, preprocess, pipelines	`scikit-learn` (`train_test_split`, `ColumnTransformer`, `Pipeline`)	new
The two models	`scikit-learn` (`LinearRegression`, `RandomForestRegressor`)	new
Evaluation metrics	`scikit-learn.metrics` (RMSE, MAE, R^2)	new
AI assistant	`groq`	new
Optional demo UI	`ipywidgets` (in-notebook)	optional

If most of the table says "new" — good. That is exactly your learning gap, and closing it is the point of the internship. We build it together, one module a week, and the Build Spec gives you runnable starter code for every step so you are never guessing.

04

The team — how you'll work together

There are **five of you**. You build **one notebook, together, in order** — Module 0 through Module 9 (plus an optional Module 10), a module or two at each weekly meeting. Nobody works on a separate copy: whoever is sharing their screen types while the rest follow and help, and the notebook lives in your one shared GitHub repo so there is always a single current version.

The part you *do* divide up is the **report** — writing is naturally a per-person job. Split the chapters so everyone owns a piece:

REPORT CHAPTER	WHAT IT COVERS
Background & Data	the problem, the dataset, how you cleaned it and chose the 14 features (M1–M2)
EDA & Methodology	the exploration charts + how the two models work, in plain English (M3–M6)
Results & Evaluation ★	the comparison table, charts, feature importance — which model won and why (M7)
AI Assistant & Honesty	the LLM layer, the describe-a-house feature, and the "model predicts, LLM explains" discussion (M8)
Abstract, Intro, Conclusion & Assembly	framing, limitations, future work — plus repo/Colab ops and final assembly

Two rules that keep you safe: build the notebook **in order** (each module needs the one before it), and **commit to GitHub every week** so you never lose work. Stuck? Say so early in the chat.

05

Weekly plan & meeting calendar

We meet on Microsoft Teams — the **kickoff is Tue 23 Jun**, then **every Tuesday, 7:00–9:00 PM IST** through submission (6 meetings total). Each meeting reviews a concrete deliverable and your blockers. **Come to every meeting with three things:** what you did, what's blocking you, what's next.

> The **module codes** below (M0–M9) refer to the build modules in the **Build Specification → Weekly Plan**, which gives the exact build → test-run → demo steps for each sprint. This calendar and that plan are the same schedule, two views.

MEETING	DATE · 7–9 PM IST	WE REVIEW / DECIDE	DELIVERABLE DUE AT THIS MEETING
1 · Kickoff	Tue 23 Jun	Meet, finalise project, roles, setup plan	— (Week-1 work assigned tonight)
2	Tue 30 Jun	Is everyone set up? Data loading?	M0 · M1 · M2 **Colab + GitHub repo** set up · `df_raw` loads (2930×82) · `X` is 2925×14
3	Tue 7 Jul	EDA + first model	M3 · M4 · M5 Two EDA charts · train/test split · Linear Regression baseline R^2 (~0.88)
4	Tue 14 Jul	Second model + pick the winner ★	M6 · M7 Random Forest · comparison table + charts · `final_model` chosen — you can say which won and why
5	Tue 21 Jul	The app + AI layer + scope freeze	M8 · M9 M10? Predict single + batch · explain-a-price + describe-a-house · **feature freeze** · decide on optional UI
6 · Final	Tue 28 Jul	Report draft + last review	optional M10 report Notebook clean top-to-bottom · each owner's draft section · dry-run
Submit	Fri 31 Jul	—	Final report submitted to ISI

Timeline reality: kickoff (Tue 23 Jun) to deadline (31 Jul) is **≈ 5.5 working weeks across 6 meetings** — tight, because we started a little later than planned. That's exactly why scope is lean and why you write the report *throughout*, not at the end. No new ideas after Meeting 5's feature freeze.

06

Reading list

Grouped by topic. Start with the new areas — that's where your learning gap is. Everything links to **official docs or established tutorials**.

Regression — the concept

- scikit-learn — LinearRegression
- scikit-learn — RandomForestRegressor

Preprocessing & pipelines new

- scikit-learn — Pipeline
- scikit-learn — ColumnTransformer (compose)
- scikit-learn — Preprocessing (scaling, one-hot)

Evaluation

- scikit-learn — Model evaluation (RMSE, MAE, R^2)

The dataset

- Kaggle — House Prices: Advanced Regression Techniques (the same Ames data + a data dictionary)

LLM API new

- Groq — Quickstart · groq-python library · Groq API Cookbook

Tools familiar

- **Google Colab** — Welcome to Colaboratory (Google's own intro notebook)

07

The Project Report

The report is the **primary assessed deliverable** — and you should write it *as you go*, not in the last week.

> **Format pending from ISI.** ISI uses its own report format, which we haven't received yet. The official template will be confirmed in **Meeting 2 (30 Jun)**. Until then, use the provisional outline below only as a guide for *what content to capture* — not the final structure.

Provisional outline (start gathering this content now): Abstract · Introduction · Background (regression, Linear Regression vs Random Forest) · Data & EDA · Methodology · Experiments & Evaluation *(your strongest section)* · Results & Discussion (incl. feature importance) · The valuation app & its use cases · The AI Assistant & the honesty guardrail · Limitations & Future Work · Conclusion · References · Appendix (the notebook).

Divide the chapters across the team (see §04) so everyone owns a piece. Agree who takes what in Meeting 2.

Where this can go next (great Future-Work material)

Same problem, your own data: the whole pipeline isn't tied to Ames, Iowa. After the project, swap in an **Indian / Bengaluru housing dataset** (point Module 1 at it, adjust the feature names) and re-run everything from there — a great way to take the method somewhere you care about. (Indian data is messier, which is why we learn on the clean Ames file first.)

Same recipe, other problems: this exact workflow — load → features → split → train two models → compare → predict → explain — also predicts **customer churn, employee attrition, loan default,** and more. Those are *yes/no* questions (classification), so you'd swap the model family and the metrics (accuracy/precision instead of R^2/RMSE), but the shape is identical. Learn it once here, reuse it everywhere — say so in your Future Work.

08

Tools, setup & where things live

Where everything lives

TOOL	USED FOR
Microsoft Teams (our group community)	**All communication, the weekly meetings, and file sharing** — the dataset, the report, meeting notes. This is our home base.
GitHub (one shared repo)	**All code** — the notebook + `requirements.txt`. The team **creates one repo in Week 1** and commits to it every week.
Google Colab	The platform you **build on** — a free, browser-based Jupyter notebook. Nothing to install.

New to Google Colab? Start here

- Welcome to Colaboratory — Google's own intro notebook (it opens automatically when you visit Colab).

Colab is just Jupyter in the browser — the same kind of notebooks you used in the ML/DL sessions, but with nothing to install and an identical environment for all five of you. `pandas`, `numpy`, `scikit-learn` and `matplotlib` come pre-installed; you'll `pip install groq` in a cell for the AI layer.

Day-1 checklist (do all of this in Week 1)

1. **Join the Teams community** — everyone (your mentor posts the link).
2. **Google Colab** — everyone signs in with a Google account at colab.research.google.com.
3. **Create ONE shared GitHub repo** for the team, add a `requirements.txt`, give all five access. **All code lives here.**
4. **Free Groq key** — sign up at console.groq.com (no credit card), create an API key, store it as a Colab secret / in a `.env` file. Never paste keys into shared code or GitHub. *(Only needed for M9 — you can start without it.)*
5. **Smoke test** — one Colab cell that imports `pandas` and `sklearn` and loads the CSV. Works → environment ready.

Security: a leaked API key can be abused. Keep keys out of GitHub — use Colab Secrets or a `.gitignore`d `.env` file.

09

How we work together

- **Weekly 2-hour touchpoint with the mentor.** Come prepared: what you did, what's blocking you, what you'll do next. The agenda for each week is in §05.
- **Definition of "done" each week** = code committed to GitHub + a one-paragraph note of progress + blockers named. "Almost working" on a laptop doesn't count — push it.
- **Say when you're stuck.** A blocker raised on day 2 is a 10-minute fix; the same blocker hidden until the touchpoint costs a week. There is no penalty for asking.
- **Be honest in the report.** "The Random Forest only beat Linear Regression by a little, and here's where it still struggles" is a *strong* result, not a failure. Examiners reward honest evaluation over inflated claims.

Remember the goal: two models you understand deeply, an honest comparison, and a report that explains it clearly — plus an AI layer that explains, never invents. Depth beats shine.